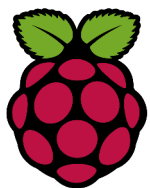


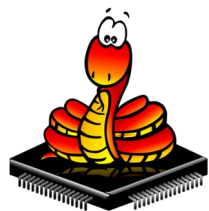


Atelier Raspi

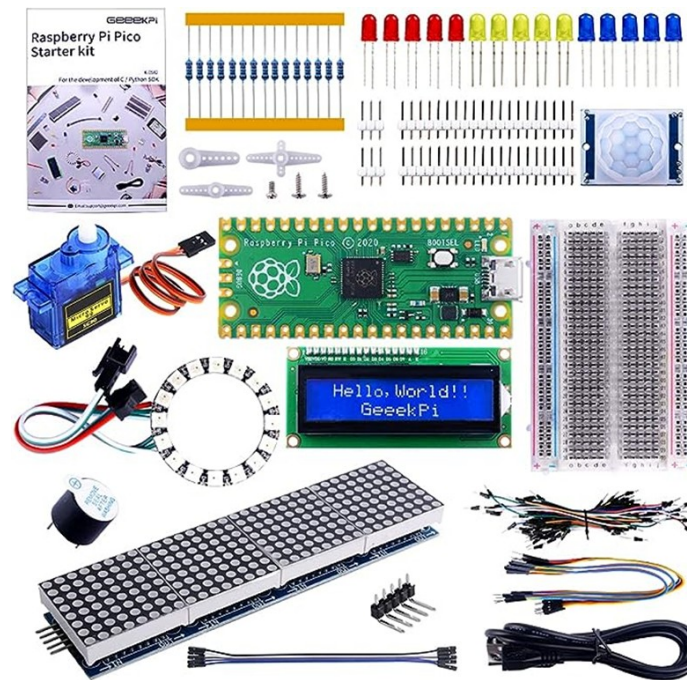
Atelier N°20 La Base de Jeux



Logo du Raspberry Pico

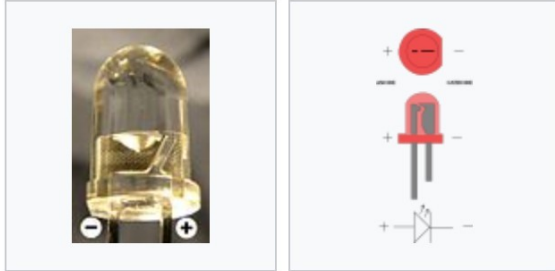


Logo du MicroPython



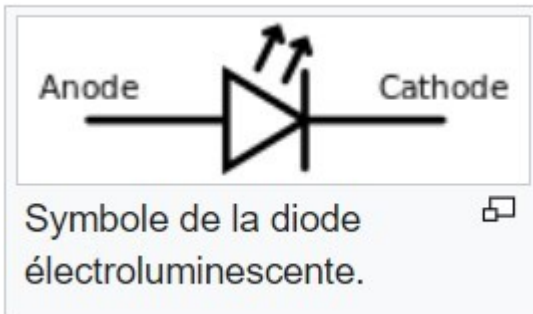
L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

La LED



Gros-plan d'une diode électroluminescente.

L'anode et la cathode d'une LED. Les signes indiquent la polarisation (courant conventionnel) lorsque la diode est utilisée en sens direct.



📺 C'est quoi une LED ?

Une **LED** (Light Emitting Diode), ou **DEL** en français (Diode Électroluminescente), est un composant électronique qui crée de la lumière quand le courant électrique le traverse. Elle consomme très peu d'énergie !

Les deux règles d'or de la LED :

- **Le sens du courant** : Une LED a un sens. Le courant doit entrer par la patte la plus longue.
- **La protection** : Une LED est fragile. Elle a toujours besoin d'une **résistance** pour ne pas griller.

Le test de la LED

On fait clignoter une LED en la branchant sur le Raspberry Pi Pico !

Un petit programme pour bien comprendre !

Le principe

On veut faire clignoter une LED :

- 1 on l'allume pendant 3 secondes,
- 2 on l'éteint pendant 2 secondes,
- 3 et on recommence !

On va utiliser la sortie PIN 6 du Raspberry Pi Pico pour commander la LED.

Le matériel

Raspberry Pi Pico

LED (sur la broche 6)

Le code

```
1 import time
2 from machine import Pin
3 #On choisit le Pin sur lequel on va monter la LED et on lui dit que c'est une sortie
4 LED = Pin(6, Pin.OUT)
5
6
7 #On donne la valeur 1 à la sortie pour allumer la LED
8 LED.value(1)
9 #On lui dit de rester allumée pendant 3 secondes
10 time.sleep(3)
11
12 #On lui dit de s'éteindre
13 LED.value(0)
14 #On lui dit de rester éteinte pendant 2 secondes
15 time.sleep(2)
```

Le code expliqué

- 1 On importe les modules nécessaires.
`import time`
`from machine import Pin`
- 2 On indique sur quelle broche on connecte la LED.
`LED = Pin(6, Pin.OUT)`
- 3 On allume la LED (valeur 1).
`LED.value(1)` → LED allumée
- 4 On attend 3 secondes.
`time.sleep(3)`
- 5 On éteint la LED (valeur 0).
`LED.value(0)` → LED éteinte
- 6 On attend 2 secondes.
`time.sleep(2)`

La LED en détails

Anode (+) Cathode (-)

La LED fonctionne comme une diode : elle ne laisse passer le courant que dans un sens. C'est pour cela qu'il faut bien la brancher !

Dans ce code, on utilise `LED.value(1)` pour l'allumer et `LED.value(0)` pour l'éteindre.

Le résultat

3 secondes LED allumée ✓

2 secondes LED éteinte ✓

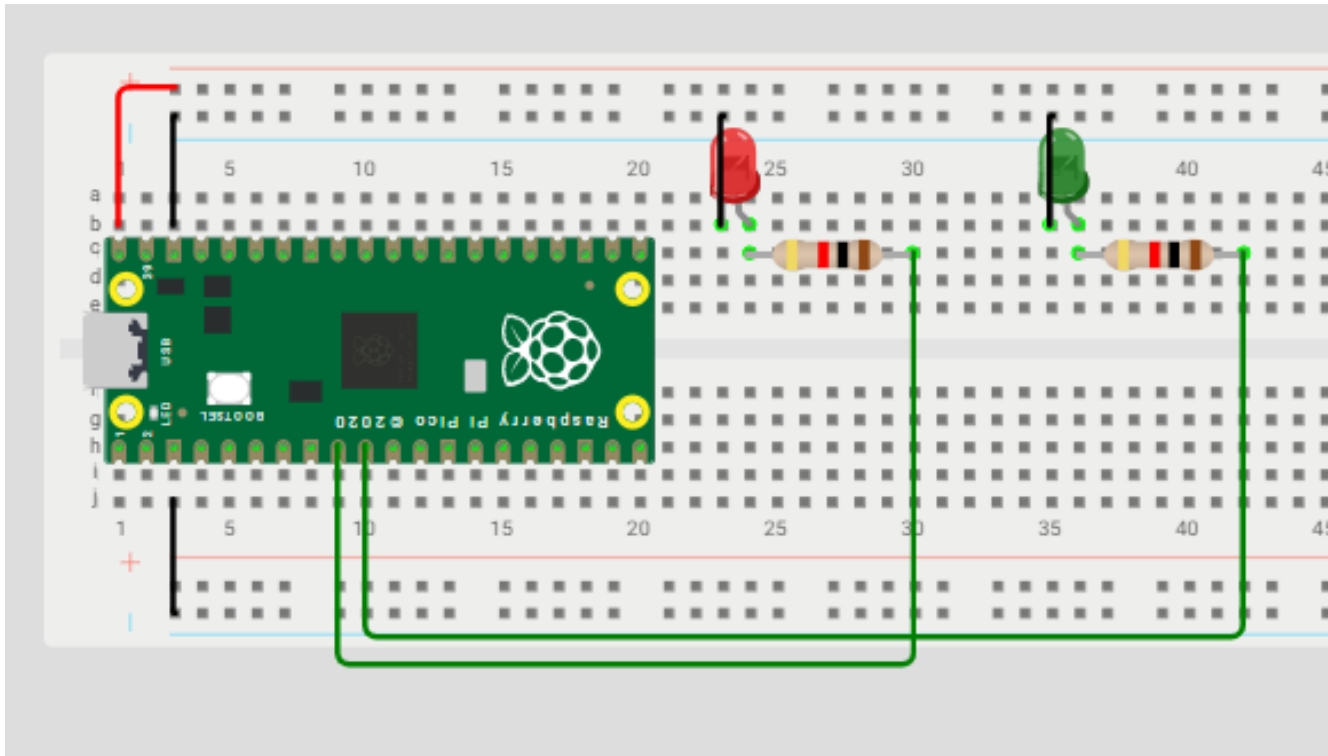
Et ça recommence !

À retenir

- GPIO 6 : la sortie qui commande la LED
- `value(1)` : allume la LED
- `value(0)` : éteint la LED
- `sleep(3)` : attend 3 secondes
- `sleep(2)` : attend 2 secondes

Simple, mais très utile pour la suite ! 😊

Câblage des 2 LEDs



Composant	GPIO
Led 1 (rouge)	6
Led 2 (verte)	7

Code couleur des câbles :

- Vcc : Rouge / Orange
- Gnd : Noir / Marron
- Données : Vert / Bleu / Jaune / Violet

Test des LEDs

```
1 # on a besoin des fonctions "time" et "Pin" (qui est dans "machine") donc on les importe
2 import time
3 from machine import Pin
4
5 #On choisit le Pin sur lequel on va monter la LED et on lui dit que c'est une sortie
6 #(pas une entree de signal) c'est a dire que la sortie PIN va fournir l'energie pour commander la LED
7 LED = Pin(6, Pin.OUT)
8 #On donne la valeur 1 à la sortie pour allumer la LED
9 LED.value(1)
10 #On lui dit de rester allumée pendant 3 secondes
11 time.sleep(3)
12 #On lui dit de s'eteindre
13 LED.value(0)
14 #On lui dit de rester éteinte pendant 2 secondes
15 time.sleep(2)
16
```



Le bouton poussoir

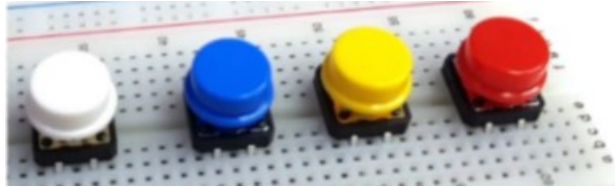
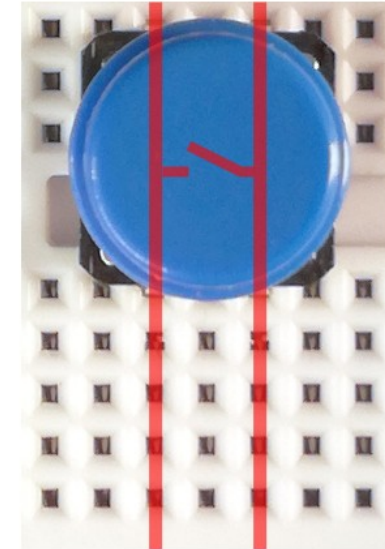


Schéma électrique interne :



Qu'est-ce que c'est ?

Le bouton poussoir est un petit interrupteur.

Quand on appuie dessus, il envoie un signal au Raspberry Pi Pico. Quand on le relâche, le signal s'arrête.



C'est un composant très simple, mais il permet de faire plein de choses : appuyer pour jouer, valider, choisir...

Comment ça marche ?



Appuyé

Le contact se ferme et le signal passe à 0 (niveau bas).



Relâché

Le contact est ouvert et le signal est à 1 (niveau haut).

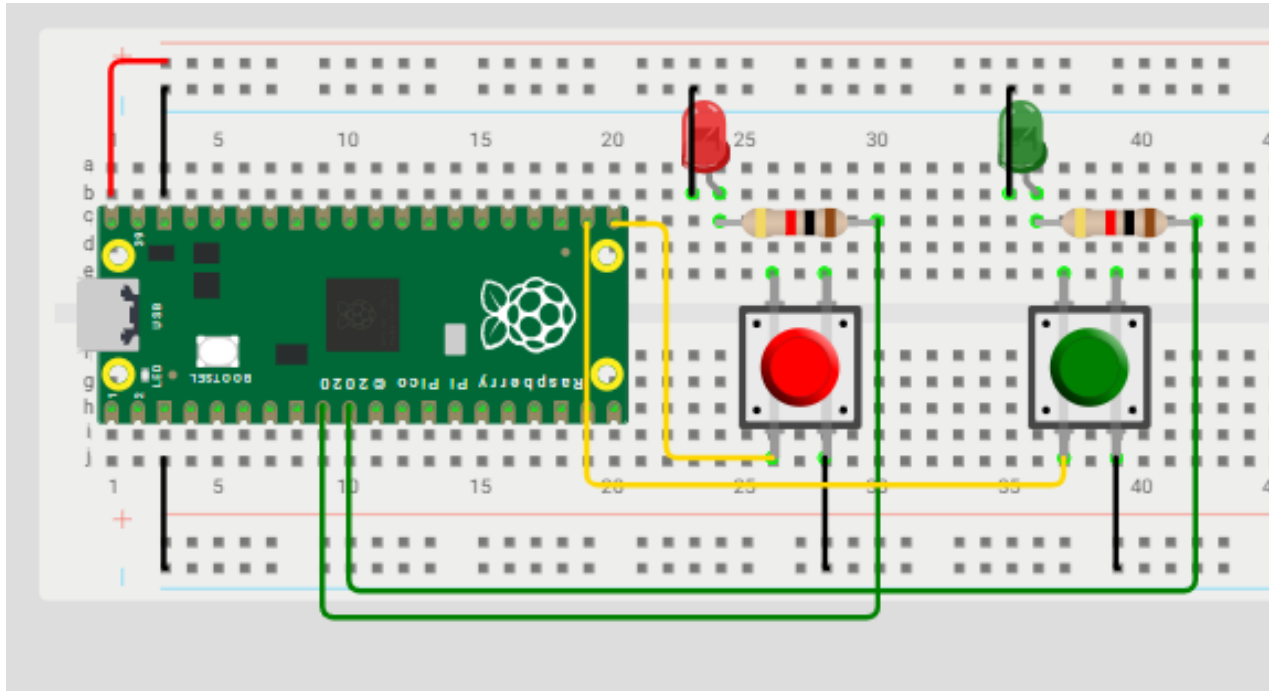
À retenir

- 1 On déclare le bouton avec `Pin(16, Pin.IN, Pin.PULL_UP)`.
- 2 On lit son état avec `bouton.value()`.
- 3 Si la valeur est 0, le bouton est appuyé.
- 4 Si la valeur est 1, il est relâché.



Un appui = un signal !

Câblage des 2 BPs

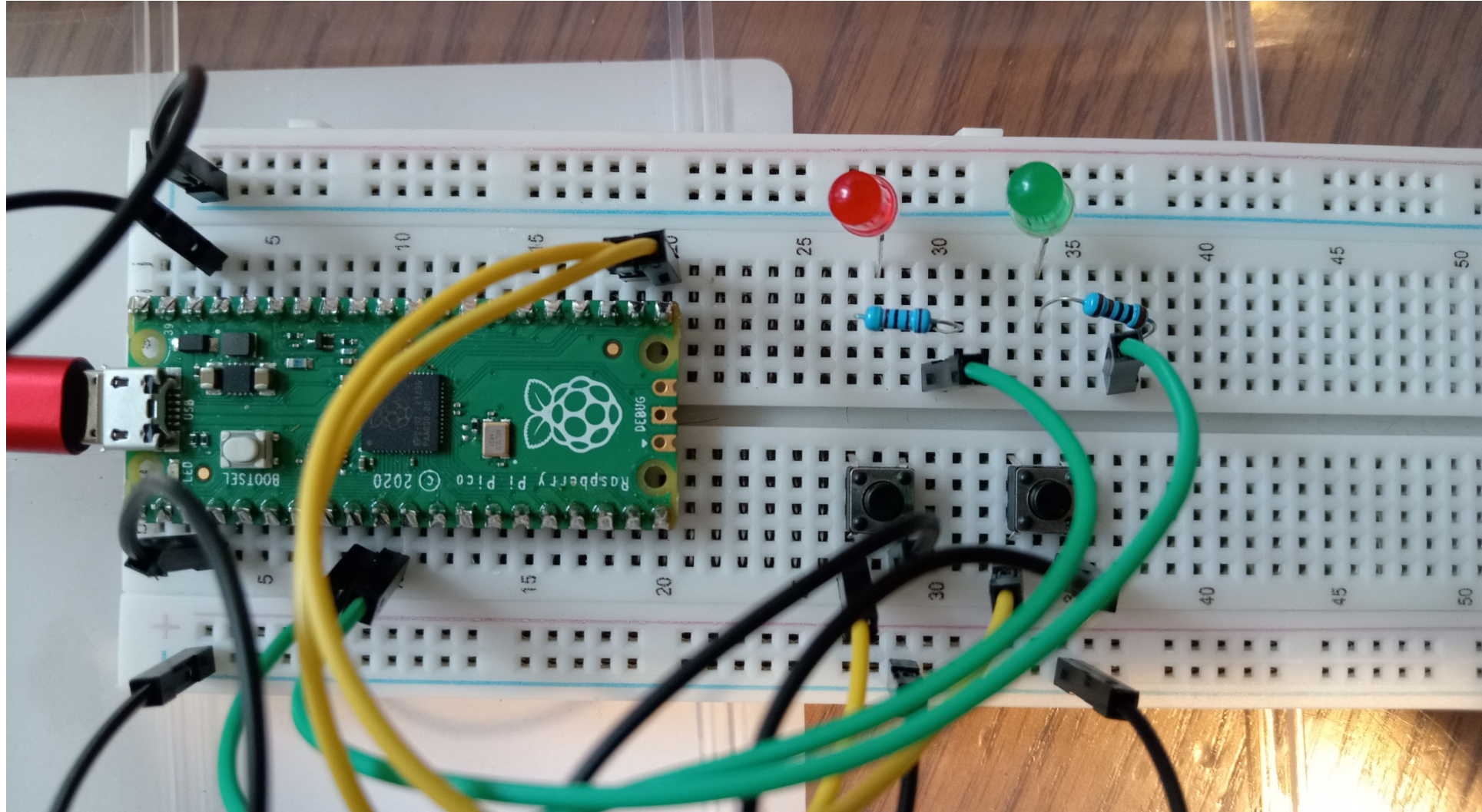


Composant	GPIO
BP 1 (rouge)	16
BP 2 (vert)	17

Test des BPs

```
1 from machine import Pin
2 import time
3
4 bouton = Pin(16, machine.Pin.IN, Pin.PULL_UP)
5
6 while True:
7     if bouton.value() == 0:
8         print("Bouton Appuyé")
9     else:
10        print("Bouton Relaché")
11        time.sleep_ms(50)
12 |
```

Montage final



Le ruban Neopixel

Le ruban NeoPixel

Une LED... mais beaucoup plus évoluée !
Un NeoPixel est une LED RGB programmable.

Contrairement à une LED classique, on peut choisir :

- sa couleur ;
- sa luminosité ;
- quelle LED du ruban doit s'allumer.

Notre ruban contient 2 NeoPixels.

Ruban NeoPixel

LED 0 → ●

LED 1 → ●

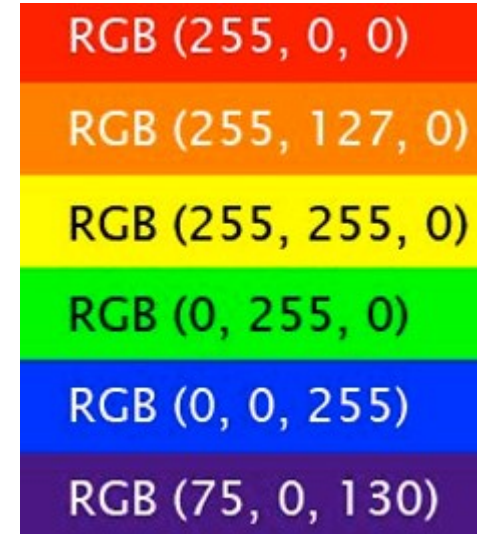
Comment commander le ruban ?

Nous utilisons la bibliothèque :

```
<> Python
from neopixel import Neopixel
```

Puis nous indiquons que notre ruban possède 2 LEDs et qu'il est connecté au GPIO 13 :

```
<> Python
NUM_LED = 2
leds = Neopixel(NUM_LED, 0, 13, "GRB")
```



Chaque LED possède un numéro

Le programme peut commander chaque LED individuellement.

Pour notre ruban :



Par exemple :

```
<> Python
leds.set_pixel(0, couleur_rouge)
```

→ La LED 0 devient rouge.

```
<> Python
leds.set_pixel(1, couleur_bleu)
```

→ La LED 1 devient bleue.

Afficher une couleur

On définit d'abord une couleur :

```
<> Python
couleur_bleu = (0, 0, 255)
```

Puis on choisit la LED :

```
<> Python
leds.set_pixel(0, couleur_bleu)
```

Et enfin, on envoie l'information au ruban :

```
<> Python
leds.show()
```

Une NeoPixel peut afficher toutes sortes de couleurs

Chaque couleur est composée de trois valeurs :

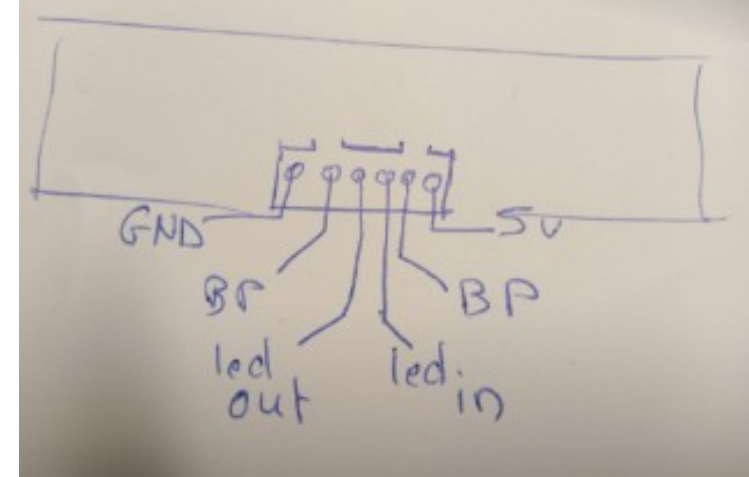
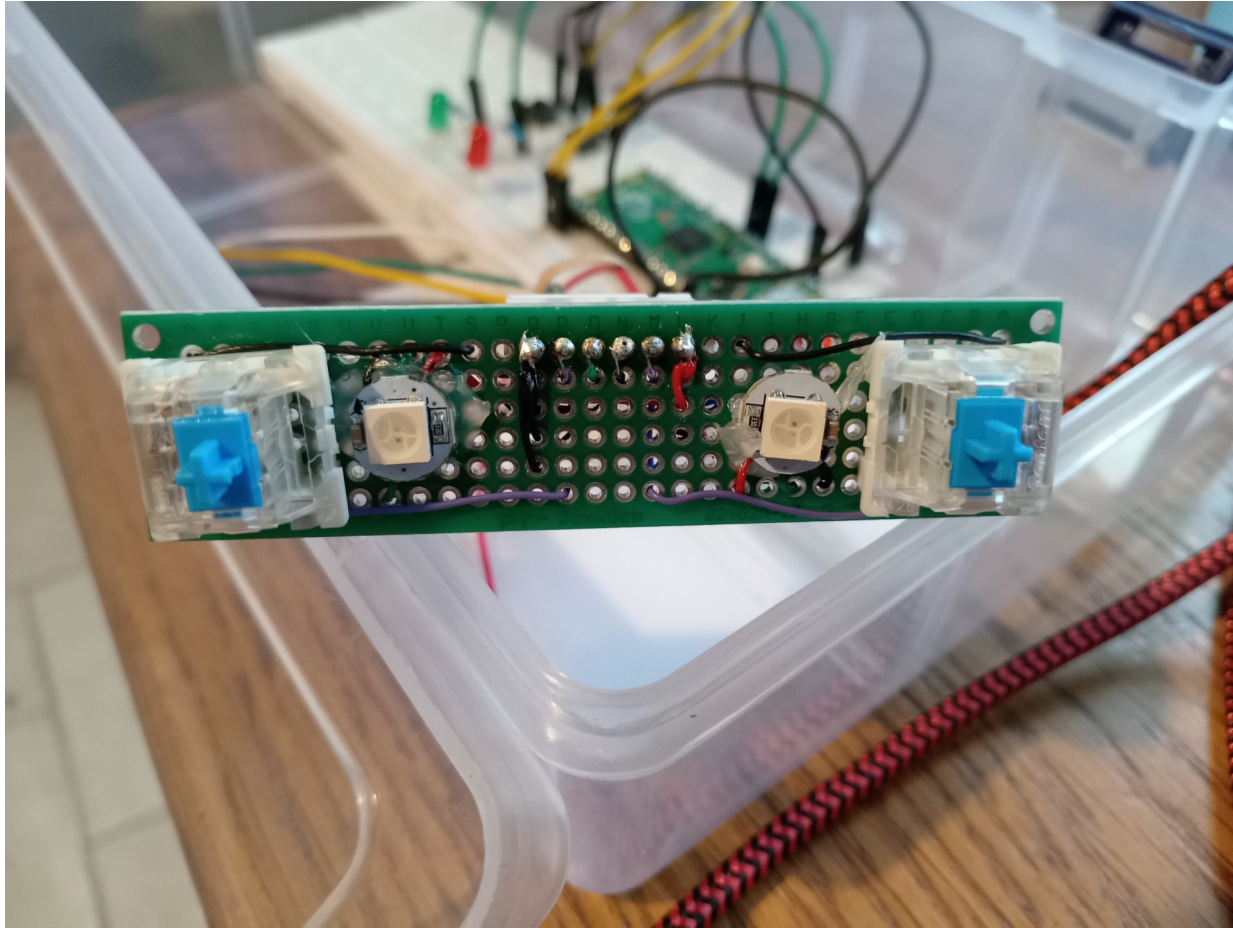
Rouge – Vert – Bleu

On appelle cela le système RGB.

(255, 0, 0)	→	● Rouge
(0, 255, 0)	→	● Vert
(0, 0, 255)	→	● Bleu
(255, 255, 0)	→	● Jaune
(255, 0, 255)	→	● Violet
(0, 0, 0)	→	● Éteint

Chaque valeur peut aller de 0 à 255.

Câblage du module Bouton + Neopixel



Composant	GPIO
BP 3 (fil noir)	18
BP 4 (fil vert)	19
Neopixel (fil jaune)	13

Test du « ruban » Neopixel

```
1 from neopixel import Neopixel
2 import time
3
4 # declaration du ruban de leds
5 NUM_LED = 2
6 leds = Neopixel(NUM_LED, 0, 13, "GRB")
7
8 # on definit la luminosite des leds / 255
9 leds.brightness(15)
10 # on eteint toute les leds
11 leds.clear()
12 # on affiche
13 leds.show()
14
15
16 while True:
17     # on allume toutes les leds en rouge pendant 1 sec
18     leds.fill((255,0,0))
19     leds.show()
20     time.sleep(1)
21     # on allume toutes les leds en vert pendant 1 sec
22     leds.fill((0,255,0))
23     leds.show()
24     time.sleep(1)
25     # on allume toutes les leds en bleu pendant 1 sec
26     leds.fill((0,0,255))
27     leds.show()
28     time.sleep(1)
29
```

```
1 from neopixel import Neopixel
2 import time
3
4 # declaration du ruban de leds
5 NUM_LED = 2
6 leds = Neopixel(NUM_LED, 0, 13, "GRB")
7
8 # declaration de quelques couleurs
9 couleur_rouge = (255,0,0)
10 couleur_vert = (0,255,0)
11 couleur_bleu = (0,0,255)
12 couleur_noir = (0,0,0)
13
14 # on definit la luminosite des leds
15 leds.brightness(15)
16 # on eteint toute les leds
17 leds.clear()
18 # on affiche
19 leds.show()
20
21 while True:
22     # on allume tour a tour les leds en bleu
23     for i in range(0, NUM_LED):
24         leds.set_pixel(i, couleur_bleu)
25         leds.show()
26         time.sleep(1)
27     time.sleep(1)
28     # on allume tour a tour les leds en vert
29     for i in range(0, NUM_LED):
30         leds.set_pixel(i, couleur_vert)
31         leds.show()
32         time.sleep(1)
33     time.sleep(1)
34     #leds.clear()
35
```

Le Jeu de Simon simplifié – Version LEDs

🎮 Jeu 1 — Le Simon à 2 couleurs

Le principe

Le jeu mémorise une séquence de couleurs et l'allonge à chaque niveau.

1. 💡 Une LED s'allume : rouge ou verte
2. 🧠 Le joueur doit mémoriser la séquence.
3. 🔄 Il reproduit la séquence avec les deux boutons.
4. ✅ S'il réussit, une nouvelle couleur est ajoutée.
5. ❌ S'il se trompe, la partie s'arrête et le score est affiché.
6. 🔄 Une nouvelle partie recommence avec une séquence vide.

💻 Ce que l'on va apprendre

Avec ce jeu, on découvre notamment :

- les listes pour mémoriser la séquence ;
- les boucles `for` et `while` ;
- les conditions `if / else` ;
- les fonctions pour organiser le programme ;
- le module `random` pour choisir une couleur au hasard ;
- la lecture de l'état d'un bouton ;
- le pilotage des LEDs ;
- la gestion d'un niveau et d'un score.

🌟 Une idée importante

Le programme est découpé en petites fonctions :

```
allumer_led()
↓
attendre_bouton()
↓
afficher_sequence()
↓
JEU SIMON
```



Pourquoi faire des fonctions ?

Pour éviter de répéter le même code et rendre le programme **plus facile à comprendre et à modifier**.

🚀 Pour aller plus loin

Certaines fonctions du jeu sont volontairement laissées vides :

```
</> Python

def erreur():
    pass

def victoire():
    pass
```

À vous de les programmer !

Par exemple, que pourrait faire le jeu lorsqu'un joueur réussit un niveau ?
Et lorsqu'il se trompe ?

Le Jeu de Simon simplifié – Version Neopixel

Évolution du Simon : les LEDs deviennent des NeoPixels

Même jeu, nouvelles possibilités !

Dans la première version, le jeu utilise :

- 2 LEDs classiques
- 2 boutons
- 1 couleur par LED

Dans la nouvelle version, les deux LEDs sont remplacées par 2 LEDs RGB NeoPixel.

Chaque LED du ruban peut être commandée **indépendamment** et peut afficher différentes couleurs.

Ce qui change

Version 1

GPIO 6 →  LED rouge
GPIO 7 →  LED verte

Version 2

GPIO 13 →  NeoPixel 1
 NeoPixel 2

Un seul GPIO permet de commander les deux NeoPixels.

Le jeu reste identique

Le joueur doit toujours :

1. regarder la séquence lumineuse ;
2. mémoriser les couleurs ;
3. reproduire la séquence avec les boutons ;
4. progresser niveau après niveau ;
5. recommencer s'il se trompe.

La logique du jeu ne change donc presque pas.

Ce sont principalement les fonctions qui contrôlent les LEDs qu'il faut modifier.

Le jeu de reflexe – Version LEDs

Le jeu de reflexe – Version Neopixel